P01 Basic Movement

CRE132 worksheet

What you'll make

The start of every platformer: a player who runs left and right and jumps, but only when standing on the ground. You'll write one script, `PlatformerController`, that puts together three things you've met before: moving a Rigidbody2D (C05), checking for ground (C06) and reading the Input System (C11).

By the end you'll know why input is read in `Update` but physics happens in `FixedUpdate`, how a small circle at the player's feet finds the ground, and which Rigidbody 2D settings make a player feel solid.

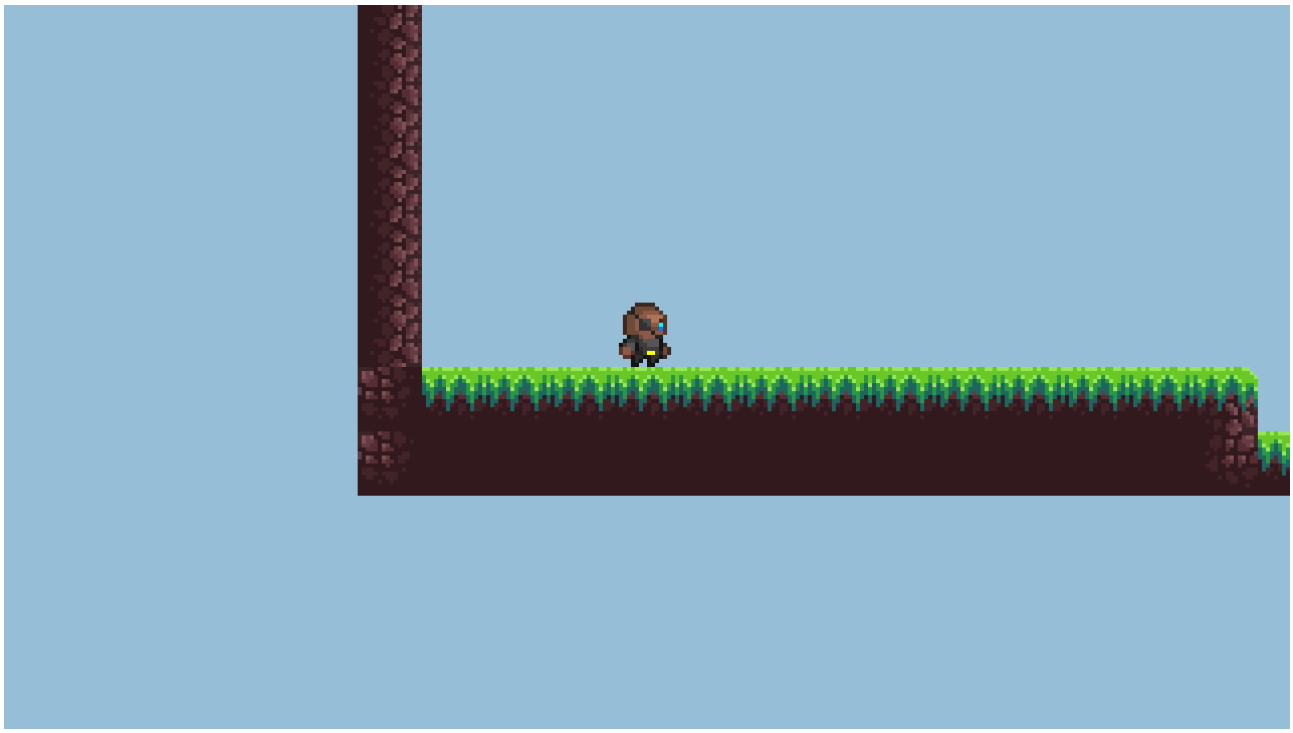


Figure 1: The start of the level: the player on the ground, with floating platforms, a dip and steps further along.

Before you start

1. Unzip `CRE132_P01_BasicMovement_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P01_BasicMovement_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P01_BasicMovement`.
3. Press **Play**. The player drops onto the ground and stands there: nothing moves yet, because the player has no controller script. Press Play again to stop.

The scene has the **Player** (with a child called **Ground Check** at its feet), the level on a **Grid**, and a **CM Follow Camera**. You'll write the script that moves the Player.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The PlatformerController script

Make a folder `Assets/_game/Scripts/P01_BasicMovement` and create a script in it called `PlatformerController`. Double-click it to open it, and replace everything with:

```
// A basic platformer controller in one script: run left and right, and jump when
// standing on the ground.
// Later samples build on it: P02 makes the jump feel better, P03 adds double jump,
// wall jump and dash.
using UnityEngine;
using UnityEngine.InputSystem;

public class PlatformerController : MonoBehaviour
{
    [Header("Movement")]
    [SerializeField] private float moveSpeed = 6f;

    [Header("Jumping")]
    [SerializeField] private float jumpSpeed = 15f;

    [Header("Ground Check")]
    // An empty child object at the player's feet. A small circle there checks for
    // ground.
    [SerializeField] private Transform groundCheck;
    [SerializeField] private float groundCheckRadius = 0.2f;
    [SerializeField] private LayerMask groundLayers;

    [Header("Input")]
    [SerializeField] private InputActionReference moveAction;
    [SerializeField] private InputActionReference jumpAction;

    private Rigidbody2D body;
    private float horizontalInput;
    private bool jumpPressed;
    private bool isGrounded;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    private void OnEnable()
    {
        moveAction.action.Enable();
        jumpAction.action.Enable();
    }

    private void OnDisable()
    {
        moveAction.action.Disable();
        jumpAction.action.Disable();
    }

    // Input is read every frame in Update, so no key press is missed.
    private void Update()
    {
        horizontalInput = moveAction.action.ReadValue<Vector2>().x;

        if (jumpAction.action.WasPressedThisFrame())
        {
```

```
            jumpPressed = true;
        }
    }

    // Physics happens in FixedUpdate.
    private void FixedUpdate()
    {
        // OverlapCircle returns the ground collider it finds, or null if the circle
        // touches nothing on groundLayers.
        Collider2D ground = Physics2D.OverlapCircle(groundCheck.position,
            groundCheckRadius, groundLayers);
        if (ground != null)
        {
            isGrounded = true;
        }
        else
        {
            isGrounded = false;
        }

        Vector2 velocity = body.linearVelocity;
        velocity.x = horizontalInput * moveSpeed;

        if (jumpPressed && isGrounded)
        {
            velocity.y = jumpSpeed;
        }
        jumpPressed = false;

        body.linearVelocity = velocity;
    }

    // Draws the ground check circle in the Scene view when the player is selected:
    // green on the ground, red in the air.
    private void OnDrawGizmosSelected()
    {
        if (groundCheck == null)
        {
            return;
        }
        if (isGrounded)
        {
            Gizmos.color = Color.green;
        }
        else
        {
            Gizmos.color = Color.red;
        }
        Gizmos.DrawWireSphere(groundCheck.position, groundCheckRadius);
    }
}
```

What's going on:

- **`[Header("...")]`** puts a bold title above a group of fields in the Inspector: Movement, Jumping, Ground Check and Input. It changes nothing in the game; it just keeps the Inspector tidy.
- **Input in `Update`, physics in `FixedUpdate`.** `Update` runs every frame, so it never misses a key press. `FixedUpdate` runs at the physics rate, which is where you should change a Rigidbody. The

`jumpPressed` bool carries the press from one to the other, and is set back to `false` once it has been used.
- **`Physics2D.OverlapCircle`** looks for a collider inside a small circle at the **Ground Check** position. The **LayerMask** means it only counts things on the **Ground** layer, so the player never counts its own collider (or a coin or a trigger) as ground. A circle as wide as the feet still finds the ground when you stand right on the edge of a platform, where a single ray from the middle (as in C06) would miss.
- **The jump** sets the upward speed (`velocity.y`) to `jumpSpeed`, but only when grounded. We change `velocity.x` and `velocity.y` and leave everything else as the Rigidbody had it.

Save the script (Ctrl+S) and go back to Unity. If the Console shows red errors, compare your code with the listing above, line by line.

2. Add it to the Player

Select the **Player** and click **Add Component > Platformer Controller**. Leave **Move Speed** at **6**, **Jump Speed** at **15** and **Ground Check Radius** at **0.2**, then fill the empty slots:

- **Ground Check:** drag the **Ground Check** child (under Player in the Hierarchy) into this slot.
- **Ground Layers:** open the drop-down and tick **Ground**.
- **Move Action:** click the small circle and pick **Player/Move** (from `CRE132Controls`).
- **Jump Action:** pick **Player/Jump** the same way.

Test it: press Play. Run with **A/D** or the arrow keys and jump with **Space** (on a gamepad: the left stick and the South button). Pressing Space in the air does nothing. While playing, select the **Player** and look at the **Scene** view: the ground check circle is green on the ground and red in the air.

3. The Player's physics settings

These are already set on the Player, but they matter as much as the script. Select the **Player** and look at its **Rigidbody 2D**:

- **Gravity Scale 3:** three times normal gravity. The jump is quicker and snappier than with 1. With Jump Speed 15 the player jumps about 3.8 tiles high.
- **Constraints > Freeze Rotation Z** ticked: without it the capsule would tip over.
- **Interpolate** set to **Interpolate:** physics runs at a different rate from drawing, so this smooths the player's movement between physics steps. Without it the following camera makes the player look jittery.
- **Collision Detection** set to **Continuous:** a fast-falling player can't pass through thin platforms.

Now look at the **Capsule Collider 2D**. Its **Material** is **No Friction** (from `Assets/_game/Materials/Physics`). Without it the player sticks to walls when you push into them in mid-air, because friction holds it there.

Test it: while playing, set the Capsule Collider 2D's **Material** to **None**, jump at a wall and keep holding towards it. The player hangs on the wall. Put **No Friction** back.

The camera is C02's: **CM Follow Camera** follows the Player with a dead zone, and its **Cinemachine Confiner 2D** uses **Camera Bounds** to stop it at the edges of the level.

Check your work

- ☐ A/D and the arrow keys run left and right, and the player stops when you let go.
- ☐ Space jumps from the ground, but not in the air.
- ☐ The ground check gizmo is green on the ground and red in the air.
- ☐ Pushing into a wall in mid-air doesn't stop you falling.
- ☐ Compare with `CRE132_P01_BasicMovement_Finished` if something behaves differently.

Key points

- **Input in `Update`, physics in `FixedUpdate`.** A bool like `jumpPressed` remembers the press until the next physics step uses it.
- **The ground check uses a LayerMask**, so only things on the Ground layer count as ground.
- **Tune in Play mode.** Move Speed, Jump Speed and the Rigidbody's Gravity Scale change the feel completely. Changes made in Play mode are lost when you stop, so write down the values you like first.

If something goes wrong

- **NullReferenceException in OnEnable:** the Move Action or Jump Action slot is empty.
- **NullReferenceException in FixedUpdate:** the Ground Check slot is empty.
- **The player runs but never jumps: Ground Layers** is set to **Nothing**, or the wrong layer. It must be **Ground**. Check the gizmo: if it stays red on the ground, the ground check can't see the floor.
- **The player can jump in mid-air:** Ground Layers includes the **Player** layer, so the player finds its own collider. Tick only **Ground**.
- **"The type or namespace name 'InputSystem' does not exist":** the `using UnityEngine.InputSystem;` line is missing or misspelt.

Try it

1. Find a **Jump Speed** and **Gravity Scale** pair that feels floaty, then one that feels heavy.
2. Move the **Ground Check** child too low, then too high, and see what breaks.
3. Make the level harder, for example a wider gap or a higher step. Open **Window > 2D > Tile Palette**, choose the **Cliffside** palette (from `Assets/_game/Tilemaps/Palettes`), and paint on the **Ground** tilemap as in C03.